

=====

Sequence Parallelism (SP) + Tensor Parallelism (TP) - Step-by-Step Example

=====

This document explains the SP-TP transition process in distributed LLM training with concrete numerical examples.

=====

SETUP: Dimensions and Distributed Configuration

=====

For clarity, we use small dimensions:

- Batch size (b) = 1
- Sequence length (s) = 4
- Hidden dimension (h) = 4
- Number of GPUs = 2 (GPU0 and GPU1)

In SP region: each GPU has (b, s/2, h) = (1, 2, 4)
In TP region: each GPU has (b, s, h/2) = (1, 4, 2)

=====

SECTION 1: INPUT TENSORS X1*, X2* (SP Region)

=====

Input is SPLIT along sequence dimension (s/2 per GPU):

GPU0 holds X1*:	GPU1 holds X2*:
Shape: (1, 2, 4) Sequence tokens 0-1: [[[1, 2, 3, 4], [5, 6, 7, 8]]]	Shape: (1, 2, 4) Sequence tokens 2-3: [[[9, 10, 11, 12], [13, 14, 15, 16]]]

Full sequence (conceptual):

Token 0:	[1, 2, 3, 4]	(on GPU0)
Token 1:	[5, 6, 7, 8]	(on GPU0)
Token 2:	[9, 10, 11, 12]	(on GPU1)
Token 3:	[13, 14, 15, 16]	(on GPU1)

=====

SECTION 2: LAYERNORM (SP Region - Independent Computation)

=====

Each GPU applies LayerNorm independently to its sequence chunk.
(Simplified LayerNorm for demonstration - just normalize to mean=0, std=1)

GPU0: Y1* = LayerNorm(X1*) GPU1: Y2* = LayerNorm(X2*)

For simplicity, assume LayerNorm outputs (scaled values):

Y1* (GPU0):	Y2* (GPU1):

```
[[[0.1, 0.2, 0.3, 0.4],  
  [0.5, 0.6, 0.7, 0.8]]]  
Shape: (1, 2, 4)
```

```
[[[0.9, 1.0, 1.1, 1.2],  
  [1.3, 1.4, 1.5, 1.6]]]  
Shape: (1, 2, 4)
```

=====

SECTION 3: "g" OPERATION (ALL-GATHER) - SP → TP Transition

=====

ALL-GATHER: Each GPU sends its data to all GPUs, all GPUs receive all data

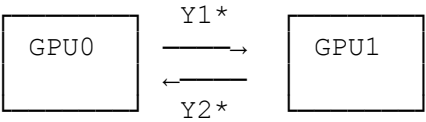
Before All-Gather:
GPU0 has: Y1* (tokens 0-1)
GPU1 has: Y2* (tokens 2-3)

After All-Gather - BOTH GPUs have full Y (b, s, h):

Y (on BOTH GPU0 and GPU1):

[[[0.1, 0.2, 0.3, 0.4],	← Token 0	(from Y1*)
[0.5, 0.6, 0.7, 0.8],	← Token 1	(from Y1*)
[0.9, 1.0, 1.1, 1.2],	← Token 2	(from Y2*)
[1.3, 1.4, 1.5, 1.6]]]	← Token 3	(from Y2*)
Shape: (1, 4, 4)		

Communication Pattern:



All-Gather complexity for data size D across P GPUs:

- Each GPU sends: D bytes
- Each GPU receives: D × (P-1) bytes
- Total bandwidth: D × (P-1) per GPU
- Time: O(D × (P-1) / bandwidth)

=====

SECTION 4: WEIGHT MATRICES A1, A2 (Column-Linear)

=====

Column-Linear: Weight matrix is SPLIT along OUTPUT dimension (columns)
Full weight A would be (h, h) = (4, 4), but split into A1 and A2.

A1 (on GPU0) - columns 0-1:

```
Shape: (4, 2)  
  
[[1, 2],  
 [5, 6],  
 [9, 10],
```

A2 (on GPU1) - columns 2-3:

```
Shape: (4, 2)  
  
[[3, 4],  
 [7, 8],  
 [11, 12],
```

```
[13, 14]]
```

```
[15, 16]]
```

Full A (conceptual) = [A1 | A2]:

```
[[1, 2, 3, 4],  
 [5, 6, 7, 8],  
 [9, 10, 11, 12],  
 [13, 14, 15, 16]]  
Shape: (4, 4)
```

SECTION 5: FIRST LINEAR (Column-Linear) - $Y \times A$

Each GPU computes: $Z_{\text{partial}} = Y \times A_{\text{local}}$

GPU0 computes: $Y \times A1$

Output shape: (1, 4, 2)

GPU1 computes: $Y \times A2$

Output shape: (1, 4, 2)

Detailed calculation for GPU0 ($Y \times A1$):

$Y \text{ (4} \times \text{4)} \times A1 \text{ (4} \times \text{2)} = Z1 \text{ (4} \times \text{2)}$

Token 0: $[0.1, 0.2, 0.3, 0.4] \times A1$
= $[0.1 \times 1 + 0.2 \times 5 + 0.3 \times 9 + 0.4 \times 13, 0.1 \times 2 + 0.2 \times 6 + 0.3 \times 10 + 0.4 \times 14]$
= $[0.1 + 1.0 + 2.7 + 5.2, 0.2 + 1.2 + 3.0 + 5.6]$
= $[9.0, 10.0]$

Token 1: $[0.5, 0.6, 0.7, 0.8] \times A1$
= $[0.5 \times 1 + 0.6 \times 5 + 0.7 \times 9 + 0.8 \times 13, 0.5 \times 2 + 0.6 \times 6 + 0.7 \times 10 + 0.8 \times 14]$
= $[0.5 + 3.0 + 6.3 + 10.4, 1.0 + 3.6 + 7.0 + 11.2]$
= $[20.2, 22.8]$

Token 2: $[0.9, 1.0, 1.1, 1.2] \times A1$
= $[0.9 \times 1 + 1.0 \times 5 + 1.1 \times 9 + 1.2 \times 13, 0.9 \times 2 + 1.0 \times 6 + 1.1 \times 10 + 1.2 \times 14]$
= $[0.9 + 5.0 + 9.9 + 15.6, 1.8 + 6.0 + 11.0 + 16.8]$
= $[31.4, 35.6]$

Token 3: $[1.3, 1.4, 1.5, 1.6] \times A1$
= $[1.3 \times 1 + 1.4 \times 5 + 1.5 \times 9 + 1.6 \times 13, 1.3 \times 2 + 1.4 \times 6 + 1.5 \times 10 + 1.6 \times 14]$
= $[1.3 + 7.0 + 13.5 + 20.8, 2.6 + 8.4 + 15.0 + 22.4]$
= $[42.6, 48.4]$

Z1 (before GeLU) on GPU0:

```
[[[9.0, 10.0],  
 [20.2, 22.8],  
 [31.4, 35.6],  
 [42.6, 48.4]]]  
Shape: (1, 4, 2)
```

↓

Z2 (before GeLU) on GPU1:

```
[[[11.0, 12.0],  
 [24.6, 27.2],  
 [38.2, 42.4],  
 [51.8, 57.6]]]  
Shape: (1, 4, 2)
```

↓

columns 0-1 of result

columns 2-3 of result

SECTION 6: GeLU ACTIVATION (Independent on Each GPU)

GeLU is applied element-wise independently on each GPU.
 $\text{GeLU}(x) \approx x \times \sigma(1.702x)$ where σ is sigmoid

For simplicity, assume $\text{GeLU}(x) \approx \max(0, x)$ (like ReLU) for positive values:

$Z1^* = \text{GeLU}(Z1)$ on GPU0:

$Z2^* = \text{GeLU}(Z2)$ on GPU1:

```
[[[9.0, 10.0],
  [20.2, 22.8],
  [31.4, 35.6],
  [42.6, 48.4]]]
Shape: (1, 4, 2)
```

```
[[[11.0, 12.0],
  [24.6, 27.2],
  [38.2, 42.4],
  [51.8, 57.6]]]
Shape: (1, 4, 2)
```

SECTION 7: WEIGHT MATRICES B1, B2 (Row-Linear)

Row-Linear: Weight matrix is SPLIT along INPUT dimension (rows)
Full weight B would be $(h, h) = (4, 4)$, but split into B1 and B2.

B1 (on GPU0) - rows 0-1:

B2 (on GPU1) - rows 2-3:

```
Shape: (2, 4)

[[0.1, 0.2, 0.3, 0.4],
 [0.5, 0.6, 0.7, 0.8]]
```

```
Shape: (2, 4)

[[0.5, 0.6, 0.7, 0.8],
 [0.9, 1.0, 1.1, 1.2]]
```

Full B (conceptual) = $\begin{bmatrix} B1 \\ B2 \end{bmatrix}$ (stacked vertically)

```
[[0.1, 0.2, 0.3, 0.4], ← B1 row 0
 [0.5, 0.6, 0.7, 0.8], ← B1 row 1
 [0.5, 0.6, 0.7, 0.8], ← B2 row 0
 [0.9, 1.0, 1.1, 1.2]] ← B2 row 1
Shape: (4, 4)
```

SECTION 8: SECOND LINEAR (Row-Linear) - $Z \times B$

Each GPU computes PARTIAL result using its portion:

GPU0: $W1 = Z1^* \times B1$
 $(1, 4, 2) \times (2, 4) = (1, 4, 4)$

GPU1: $W2 = Z2^* \times B2$
 $(1, 4, 2) \times (2, 4) = (1, 4, 4)$

Detailed calculation for GPU0 (Z1* × B1):

Token 0: [9.0, 10.0] × B1
= [9.0×0.1 + 10.0×0.5, 9.0×0.2 + 10.0×0.6, 9.0×0.3 + 10.0×0.7, 9.0×0.4 + 10.0×0.8]
= [0.9 + 5.0, 1.8 + 6.0, 2.7 + 7.0, 3.6 + 8.0]
= [5.9, 7.8, 9.7, 11.6]

Token 1: [20.2, 22.8] × B1
= [20.2×0.1 + 22.8×0.5, 20.2×0.2 + 22.8×0.6, 20.2×0.3 + 22.8×0.7, 20.2×0.4 + 22.8×0.8]
= [2.02 + 11.4, 4.04 + 13.68, 6.06 + 15.96, 8.08 + 18.24]
= [13.42, 17.72, 22.02, 26.32]

Token 2: [31.4, 35.6] × B1
= [31.4×0.1 + 35.6×0.5, 31.4×0.2 + 35.6×0.6, 31.4×0.3 + 35.6×0.7, 31.4×0.4 + 35.6×0.8]
= [3.14 + 17.8, 6.28 + 21.36, 9.42 + 24.92, 12.56 + 28.48]
= [20.94, 27.64, 34.34, 41.04]

Token 3: [42.6, 48.4] × B1
= [42.6×0.1 + 48.4×0.5, 42.6×0.2 + 48.4×0.6, 42.6×0.3 + 48.4×0.7, 42.6×0.4 + 48.4×0.8]
= [4.26 + 24.2, 8.52 + 29.04, 12.78 + 33.88, 17.04 + 38.72]
= [28.46, 37.56, 46.66, 55.76]

W1 (PARTIAL on GPU0):

[[[5.9, 7.8, 9.7, 11.6],
[13.42, 17.72, 22.02, 26.32],
[20.94, 27.64, 34.34, 41.04],
[28.46, 37.56, 46.66, 55.76]]]
Shape: (1, 4, 4)

W2 (PARTIAL on GPU1):

[[[8.1, 9.0, 9.9, 10.8],
[18.78, 21.28, 23.78, 26.28],
[29.46, 33.56, 37.66, 41.76],
[40.14, 45.84, 51.54, 57.24]]]
Shape: (1, 4, 4)

CRITICAL: W1 and W2 are PARTIAL results!
Full result W = W1 + W2 (need to sum across GPUs)

=====

SECTION 9: "g*" OPERATION (REDUCE-SCATTER) - TP → SP Transition

=====

REDUCE-SCATTER: Sum partial results (reduce) + distribute chunks (scatter)

Step 1: REDUCE (Sum W1 + W2 element-wise)

W_full = W1 + W2:

Token 0: [5.9+8.1, 7.8+9.0, 9.7+9.9, 11.6+10.8] = [14.0, 16.8, 19.6, 22.4]
Token 1: [13.42+18.78, 17.72+21.28, 22.02+23.78, 26.32+26.28] = [32.2, 39.0, 45.8, 52.6]

Token 2: [20.94+29.46, 27.64+33.56, 34.34+37.66, 41.04+41.76] = [50.4, 61.2, 72.0, 82.8]
Token 3: [28.46+40.14, 37.56+45.84, 46.66+51.54, 55.76+57.24] = [68.6, 83.4, 98.2, 113.0]

W_full (conceptual):

[[[14.0, 16.8, 19.6, 22.4],
[32.2, 39.0, 45.8, 52.6],
[50.4, 61.2, 72.0, 82.8],
[68.6, 83.4, 98.2, 113.0]]]
Shape: (1, 4, 4)

← Token 0
← Token 1
← Token 2
← Token 3

Step 2: SCATTER (Distribute along sequence dimension)

GPU0 gets: Tokens 0-1 (first s/2)
GPU1 gets: Tokens 2-3 (second s/2)

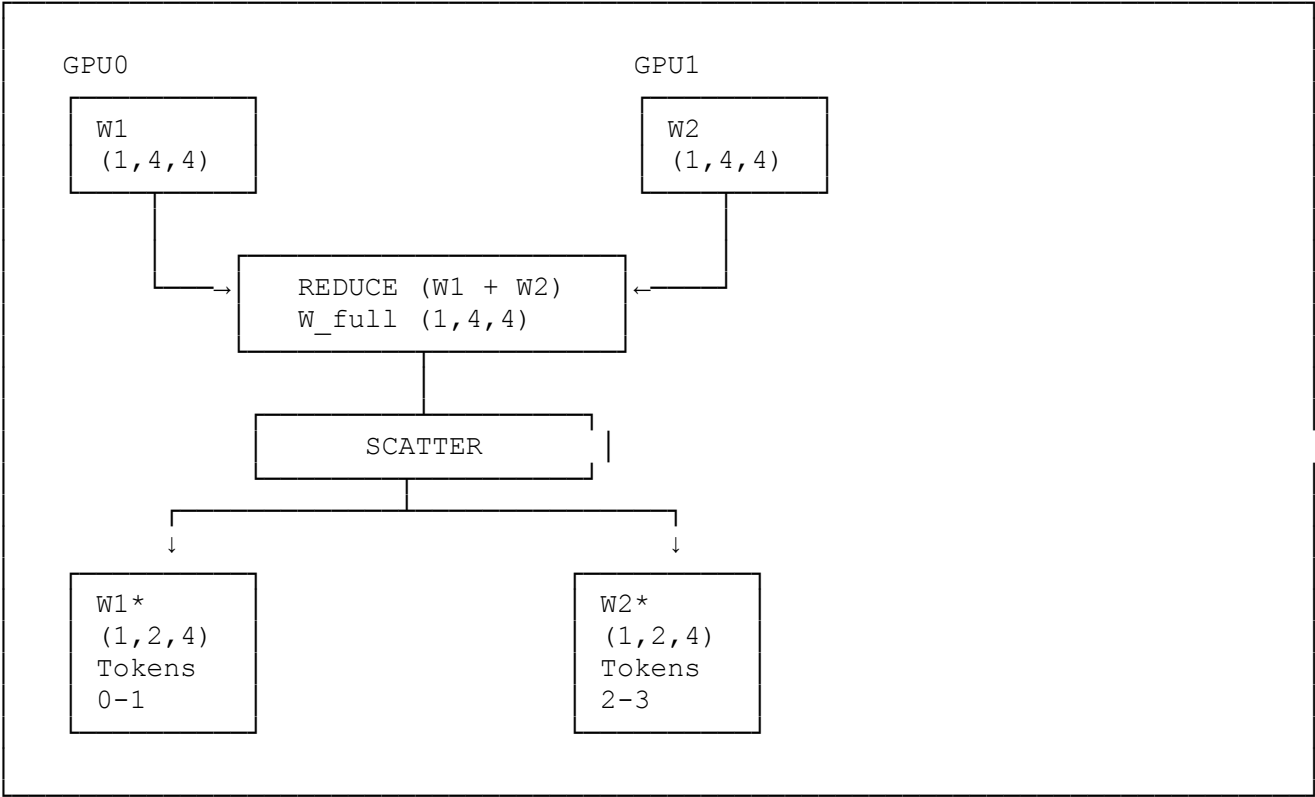
W1* (on GPU0):

[[[14.0, 16.8, 19.6, 22.4],
[32.2, 39.0, 45.8, 52.6]]]
Shape: (1, 2, 4)

W2* (on GPU1):

[[[50.4, 61.2, 72.0, 82.8],
[68.6, 83.4, 98.2, 113.0]]]
Shape: (1, 2, 4)

REDUCE-SCATTER Communication Pattern:



=====

SECTION 10: DROPOUT (SP Region - Independent)

=====

Dropout applied independently on each GPU's sequence chunk:

V1* = Dropout(W1*) on GPU0:

[[[14.0, 16.8, 19.6, 22.4],
[32.2, 39.0, 45.8, 52.6]]]
Shape: (1, 2, 4)

V2* = Dropout(W2*) on GPU1:

[[[50.4, 61.2, 72.0, 82.8],
[68.6, 83.4, 98.2, 113.0]]]
Shape: (1, 2, 4)

(Assuming dropout rate = 0 for this example)

=====

SECTION 11: SUMMARY OF ALL VALUES

=====

Variable	Value		
X1* (GPU0)	[[[1,2,3,4], [5,6,7,8]]] Shape: (1,2,4)		
X2* (GPU1)	[[[9,10,11,12], [13,14,15,16]]] Shape: (1,2,4)		
Y1* (GPU0)	[[[0.1,0.2,0.3,0.4], [0.5,0.6,0.7,0.8]]] Shape: (1,2,4)		
Y2* (GPU1)	[[[0.9,1.0,1.1,1.2], [1.3,1.4,1.5,1.6]]] Shape: (1,2,4)		
Y (both)	[[[0.1,0.2,0.3,0.4], [0.5,0.6,0.7,0.8], [0.9,1.0,1.1,1.2], [1.3,1.4,1.5,1.6]]] Shape: (1,4,4)		
A1 (GPU0)	[[1,2], [5,6], [9,10], [13,14]] Shape: (4,2)		
A2 (GPU1)	[[3,4], [7,8], [11,12], [15,16]] Shape: (4,2)		
Z1* (GPU0)	[[[9,10], [20.2,22.8], [31.4,35.6], [42.6,48.4]]] Shape: (1,4,2)		
Z2* (GPU1)	[[[11,12], [24.6,27.2], [38.2,42.4], [51.8,57.6]]] Shape: (1,4,2)		
B1 (GPU0)	[[0.1,0.2,0.3,0.4], [0.5,0.6,0.7,0.8]] Shape: (2,4)		
B2 (GPU1)	[[0.5,0.6,0.7,0.8], [0.9,1.0,1.1,1.2]] Shape: (2,4)		
W1 (GPU0)	[[[5.9,7.8,9.7,11.6], ...]]	PARTIAL	Shape: (1,4,4)
W2 (GPU1)	[[[8.1,9.0,9.9,10.8], ...]]	PARTIAL	Shape: (1,4,4)
W1* (GPU0)	[[[14.0,16.8,19.6,22.4], [32.2,39.0,45.8,52.6]]] Shape: (1,2,4)		
W2* (GPU1)	[[[50.4,61.2,72.0,82.8], [68.6,83.4,98.2,113.0]]] Shape: (1,2,4)		
V1* (GPU0)	[[[14.0,16.8,19.6,22.4], [32.2,39.0,45.8,52.6]]] Shape: (1,2,4)		
V2* (GPU1)	[[[50.4,61.2,72.0,82.8], [68.6,83.4,98.2,113.0]]] Shape: (1,2,4)		

SECTION 12: OPERATIONS EXPLAINED

1. "g" OPERATION (ALL-GATHER)

Purpose: Transition from SP (split sequence) to TP (full sequence needed)

Before: GPU0 has $Y1^*$ (tokens 0-1), GPU1 has $Y2^*$ (tokens 2-3)

After: Both GPUs have full Y (tokens 0-3)

Why needed: Column-linear ($A1, A2$) needs FULL sequence because each output column depends on ALL input elements along hidden dimension.

Formula: $Y = \text{AllGather}([Y1^*, Y2^*], \text{dim}=\text{sequence})$

2. COLUMN-LINEAR ($A1, A2$)

Purpose: First linear layer in MLP/FFN, typically expands hidden dim

Weight split: $A = [A1 \mid A2]$ (concatenate along columns/output dim)

Computation:

GPU0: $Z1 = Y \times A1 \rightarrow$ gets columns 0 to $h/2-1$ of output

GPU1: $Z2 = Y \times A2 \rightarrow$ gets columns $h/2$ to $h-1$ of output

Full result: $Z = [Z1 \mid Z2]$ (if concatenated)

But we keep $Z1, Z2$ separate (this IS the TP split)

Key insight: Each GPU computes COMPLETE rows but PARTIAL columns

3. ROW-LINEAR ($B1, B2$)

Purpose: Second linear layer in MLP/FFN, typically projects back to h

Weight split: $B = \begin{bmatrix} B1 \\ B2 \end{bmatrix}$ (stack along rows/input dim)

Computation:

GPU0: $W1 = Z1^* \times B1 \rightarrow$ PARTIAL result (only uses first $h/2$ of input)

GPU1: $W2 = Z2^* \times B2 \rightarrow$ PARTIAL result (only uses second $h/2$ of input)

Full result: $W = W1 + W2$ (SUM of partial results!)

Why SUM? Because: $Z \times B = \begin{bmatrix} Z1 & Z2 \end{bmatrix} \times \begin{bmatrix} B1 \\ B2 \end{bmatrix} = Z1 \times B1 + Z2 \times B2 = W1 + W2$

Key insight: Each GPU computes PARTIAL results that must be SUMMED

4. "g*" OPERATION (REDUCE-SCATTER)

Purpose: Transition from TP (partial results) to SP (split sequence)

Two operations combined:

1. REDUCE: Sum partial results $W1 + W2 = W_full$
2. SCATTER: Distribute W_full chunks to different GPUs

Before: GPU0 has $W1$ (partial), GPU1 has $W2$ (partial) - both (1,4,4)

After: GPU0 has $W1^*$ (tokens 0-1), GPU1 has $W2^*$ (tokens 2-3) - both (1,2,4)

Why efficient: Combines reduction and distribution in ONE operation instead of AllReduce followed by local slicing.

SECTION 13: REDUCE-SCATTER DETAILED EXPLANATION

REDUCE-SCATTER = REDUCE + SCATTER in one efficient operation

STEP-BY-STEP REDUCE-SCATTER ALGORITHM

Given: P GPUs, each with tensor of size N (total data = $P \times N$)

Goal: Sum all tensors, each GPU gets $1/P$ of result

Ring-based Reduce-Scatter ($P=2$ example):

Round 1:

GPU0 sends $W1$ [tokens 2-3] to GPU1

GPU1 sends $W2$ [tokens 0-1] to GPU0

GPU0: $W1[0-1] + \text{received } W2[0-1] = \text{partial_sum}[0-1]$

GPU1: $W2[2-3] + \text{received } W1[2-3] = \text{partial_sum}[2-3]$

Result:

GPU0 has: $\text{final_sum}[\text{tokens } 0-1] = W1^* \checkmark$

GPU1 has: $\text{final_sum}[\text{tokens } 2-3] = W2^* \checkmark$

Communication volume: Each GPU sends $N/2$ data (half their tensor)

SECTION 14: TIME AND SPACE COMPLEXITY OF REDUCE-SCATTER

REDUCE-SCATTER COMPLEXITY

Parameters:

P = number of GPUs

D = total data size (e.g., $b \times s \times h \times \text{sizeof(float)}$)

B = interconnect bandwidth

α = latency per message

TIME COMPLEXITY:

Ring algorithm:

$$\begin{aligned}\text{Communication time} &= (P-1) \times (D/P) / B + (P-1) \times \alpha \\ &= D \times (P-1)/P / B + (P-1) \times \alpha \\ &\approx D/B \quad (\text{for large } P, \text{ ignoring latency})\end{aligned}$$

Bandwidth-optimal: $O(D/B)$ - transfers $D \times (P-1)/P$ total bytes

For our example:

D = $1 \times 4 \times 4 \times 4$ bytes = 64 bytes per GPU

Each GPU sends: $64 \times (2-1)/2 = 32$ bytes

SPACE COMPLEXITY:

Input: D bytes per GPU ($W1$ or $W2$)

Output: D/P bytes per GPU ($W1^*$ or $W2^*$)

Buffer: $O(D/P)$ temporary receive buffer

Total: $O(D)$ input + $O(D/P)$ output = reduces memory by factor of P

For our example:

Before: 64 bytes (1,4,4) per GPU

After: 32 bytes (1,2,4) per GPU

Memory reduced by 50%

SECTION 15: ALL-REDUCE ALTERNATIVE (Step-by-Step)

If using ALL-REDUCE instead of REDUCE-SCATTER:

ALL-REDUCE = Every GPU gets the FULL reduced result ($W1 + W2$)

ALL-REDUCE STEP-BY-STEP

Initial State:

GPU0 has $W1$ (1,4,4)

GPU1 has $W2$ (1,4,4)

Step 1: Reduce-Scatter phase (same as before)

GPU0: partial_sum[tokens 0-1]

GPU1: partial_sum[tokens 2-3]

Step 2: All-Gather phase (additional communication!)

GPU0 sends partial_sum[0-1] to GPU1
GPU1 sends partial_sum[2-3] to GPU0

Result:

GPU0 has: W_full (tokens 0-3) = $\begin{bmatrix} [14.0, 16.8, 19.6, 22.4], \\ [32.2, 39.0, 45.8, 52.6], \\ [50.4, 61.2, 72.0, 82.8], \\ [68.6, 83.4, 98.2, 113.0] \end{bmatrix}$

GPU1 has: W_full (same as GPU0)

Then: Each GPU must locally slice to get its portion

GPU0: $W1^* = W_full[\text{tokens } 0-1]$
GPU1: $W2^* = W_full[\text{tokens } 2-3]$

ALL-REDUCE = REDUCE-SCATTER + ALL-GATHER

ALL-REDUCE transfers 2× the data of REDUCE-SCATTER!

SECTION 16: TIME AND SPACE COMPLEXITY OF ALL-REDUCE

ALL-REDUCE COMPLEXITY

Parameters: Same as before (P GPUs, D data, B bandwidth, α latency)

TIME COMPLEXITY:

Ring All-Reduce = Reduce-Scatter + All-Gather

Reduce-Scatter: $D \times (P-1)/P / B$
All-Gather: $D \times (P-1)/P / B$
Total: $2 \times D \times (P-1)/P / B$
 $\approx 2D/B$ (for large P)

ALL-REDUCE IS 2× SLOWER THAN REDUCE-SCATTER!

For our example:

Reduce-Scatter: sends 32 bytes per GPU
All-Reduce: sends 64 bytes per GPU (2×)

SPACE COMPLEXITY:

Input: D bytes per GPU
Output: D bytes per GPU (FULL result on each GPU)
Buffer: $O(D/P)$ temporary buffer

Total: $O(D)$ - NO memory reduction!

For our example:

Before: 64 bytes (1,4,4) per GPU

After: 64 bytes (1,4,4) per GPU (same!)
Then must slice locally to get (1,2,4)

SECTION 17: COMPARISON TABLE - REDUCE-SCATTER VS ALL-REDUCE

Aspect	Reduce-Scatter	All-Reduce
Output per GPU	D/P (partial)	D (full)
Communication Volume	$D \times (P-1)/P$	$2 \times D \times (P-1)/P$
Time Complexity	$O(D/B)$	$O(2D/B) = 2\times$ slower
Memory After	D/P per GPU (reduced)	D per GPU (same)
Memory Savings	$P\times$ reduction	None
Use Case	When only partial result needed (SP→TP)	When all GPUs need full result
Example (P=2, D=64B)	32B sent, 32B output	64B sent, 64B output

SECTION 18: WHY REDUCE-SCATTER IS PREFERRED FOR SP-TP TRANSITION

In Sequence Parallelism, after row-linear:

- Each GPU only needs ITS portion of the sequence
- GPU0 needs tokens 0 to $s/2-1$
- GPU1 needs tokens $s/2$ to $s-1$

Using All-Reduce would be wasteful:

1. Communicate $2\times$ the data
2. Store full result temporarily
3. Then discard half of it

Reduce-Scatter is perfect because:

1. Communicates minimal data
2. Directly produces the needed output
3. No extra memory or discarding

Formula comparison:

All-Reduce: $W_{full} = \text{AllReduce}(W1, W2)$ # all GPUs get (b,s,h)
 $W1^* = W_{full}[:, :s/2, :]$ # local slice

Reduce-Scatter: $W1^*, W2^* = \text{ReduceScatter}([W1, W2])$ # direct result!

SECTION 19: COMPLETE DATA FLOW DIAGRAM

COMPLETE SP-TP PIPELINE

GPU0

GPU1

$X1^*$
(1,2,4)

SP REGION
(split seq)

$X2^*$
(1,2,4)

LayerNorm

LayerNorm

$Y1^*$
(1,2,4)

$Y2^*$
(1,2,4)

ALL-GATHER
(SP $\rightarrow$ TP)

"g" operation

Y (1,4,4) on BOTH GPUs

$Y \times A1$
Col-Linear

TP REGION
(split hidden)

$Y \times A2$
Col-Linear

GeLU

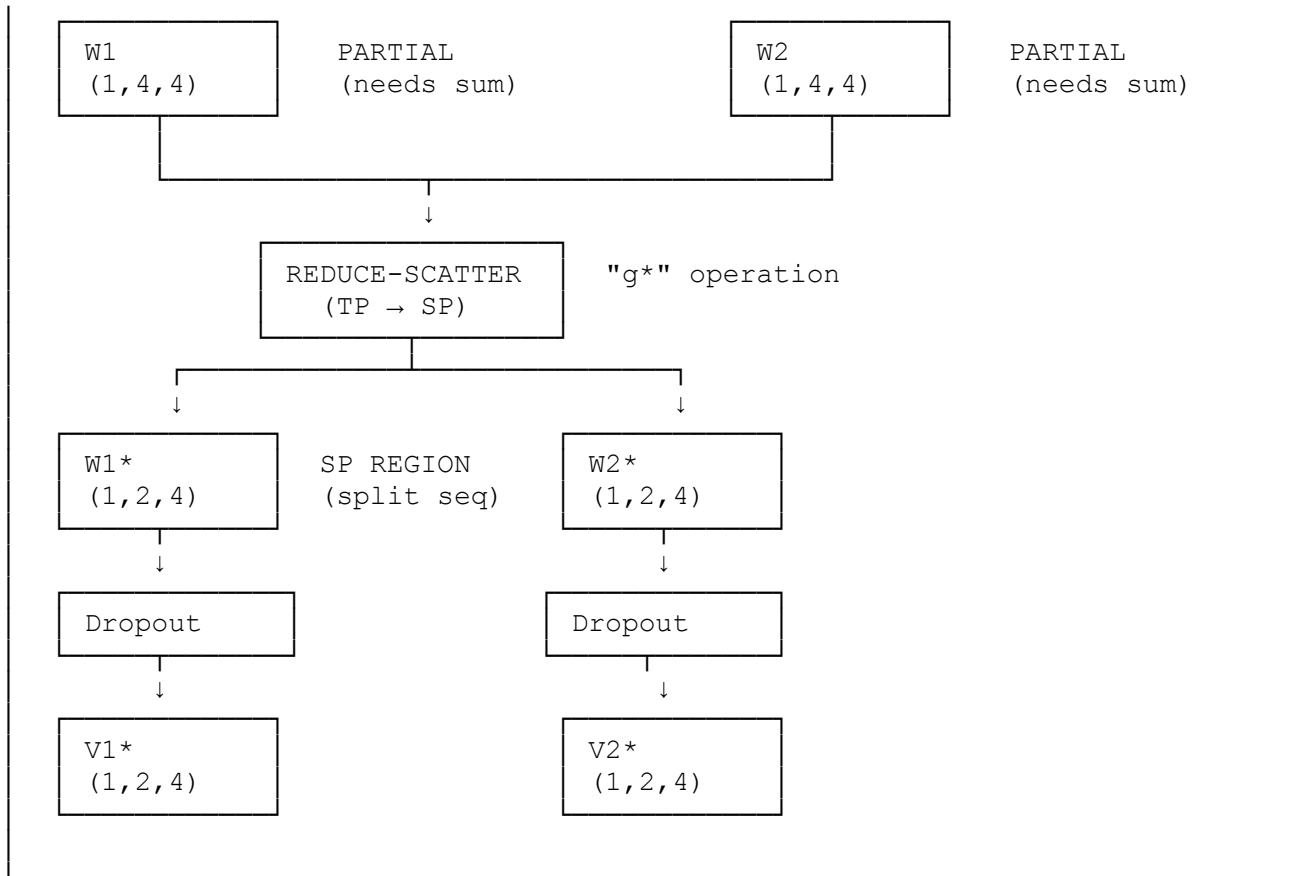
GeLU

$Z1^*$
(1,4,2)

$Z2^*$
(1,4,2)

$Z1^* \times B1$
Row-Linear

$Z2^* \times B2$
Row-Linear



SECTION 20: KEY FORMULAS SUMMARY

MATHEMATICAL FORMULAS

1. LayerNorm (independent per GPU):
 $Y1^* = \text{LayerNorm}(X1^*)$
 $Y2^* = \text{LayerNorm}(X2^*)$
2. All-Gather:
 $Y = \text{Concat}([Y1^*, Y2^*], \text{dim}=\text{sequence})$
 $Y = \text{AllGather}(Y1^*, Y2^*)$ // collective operation
3. Column-Linear (weight split along output):
 $Z1^* = \text{GeLU}(Y \times A1)$ // GPU0 computes columns 0 to h/2-1
 $Z2^* = \text{GeLU}(Y \times A2)$ // GPU1 computes columns h/2 to h-1
 Conceptually: $Z = [Z1^* \mid Z2^*]$
4. Row-Linear (weight split along input):
 $W1 = Z1^* \times B1$ // GPU0 partial result
 $W2 = Z2^* \times B2$ // GPU1 partial result
 $W_{\text{full}} = W1 + W2$ // sum needed for correctness

```

5. Reduce-Scatter:
  [W1*, W2*] = ReduceScatter([W1, W2], dim=sequence)
  W1* = (W1 + W2)[tokens 0:s/2]      // GPU0 gets first half
  W2* = (W1 + W2)[tokens s/2:s]      // GPU1 gets second half

6. Dropout (independent per GPU):
  V1* = Dropout(W1*)
  V2* = Dropout(W2*)

```

COMPLEXITY FORMULAS

For P GPUs, data size D, bandwidth B:

```

All-Gather:
  Time = D × (P-1) / B
  Output = P × D per GPU (replicated)

```

```

Reduce-Scatter:
  Time = D × (P-1) / (P × B) ≈ D/B
  Output = D / P per GPU (distributed)

```

```

All-Reduce = Reduce-Scatter + All-Gather:
  Time = 2 × D × (P-1) / (P × B) ≈ 2D/B
  Output = D per GPU (replicated)

```

=====

END OF DOCUMENT

=====